
SYSTEM ARCHITECTURE & TECHNICAL DOCUMENTATION

ROLLESTONE

(Requirement, Feasibility & DevOps Architecture)

Independent SaaS Platform (Freelance Client Project)

PREPARED BY

Md Omar Faruk

Lead Developer & System Architect

Dhaka, Bangladesh

GitHub: github.com/kaziomارش24

Email: softeng.kaziomar@gmail.com

PROJECT DETAILS

Target Market: New Zealand

Industry: Transport Sector

Architecture: Multi-tenant SaaS

Infrastructure: Docker & CI/CD

Date of Documentation: February 07, 2026

Contents

1	Introduction	4
1.1	Role of System Analyst	4
1.2	Project Overview	4
1.3	Objectives	4
1.4	Software Development Methodology	4
2	Requirement Analysis	5
2.1	Information Gathering Techniques	5
2.2	Functional Requirements	5
2.2.1	Passenger Module (Mobile App)	5
2.2.2	Driver Module (Tablet/iPad App)	6
2.2.3	Admin Panel (Web Dashboard)	6
2.3	Non-Functional Requirements & Constraints	7
3	Feasibility Study	8
3.1	Economic Feasibility (Cost & Timeline)	8
3.2	Technical Feasibility	8
3.3	Operational Feasibility	8
3.4	Schedule Feasibility	8
3.5	Behavioral Feasibility	9
3.6	System Workflow Diagram	9
4	Structured Analysis & System Design	10
4.1	Data Flow Diagram (DFD)	10
4.1.1	Level 0: Context Diagram	10
4.1.2	Level 1 DFD: Sub-system Decomposition	10
4.1.3	Level 2 DFD: "Scan & Go" Payment Logic	11
4.2	Database Architecture & Schema Design	12
4.3	Data Dictionary	13
4.3.1	Module 1: Identity & Access Management	13
4.3.2	Module 2: Fleet Operations Logic	14
4.3.3	Module 3: Financial & Transaction Systems	15
4.4	Database Architectural Decisions	16
5	Implementation Details	17
5.1	Development Environment & Configuration	17
5.1.1	Server Architecture (Docker Services)	17
5.1.2	Backend Dependencies	18
5.1.3	Nginx Reverse Proxy Configuration	18
5.1.4	Microservices Repository Structure	19
5.2	Database Implementation	19
5.2.1	Journey Management Schema	19
5.2.2	Financial Transaction Schema	20
5.3	Architectural Patterns & Code Reusability	20
5.3.1	Base Service & Abstraction	20
5.3.2	Custom Traits (File Upload & Optimization)	21
5.3.3	SaaS Tenant Resolution Logic	21
5.4	Core Backend Logic & APIs	22
5.4.1	API Routes Definition	22

5.4.2	Stripe Payment Integration	22
5.4.3	Automated Webhook Handling	23
5.5	Automated Ticketing & Geo-Spatial Logic	23
5.5.1	Scan & Go Payment Algorithm	24
5.5.2	Real-time Location Broadcasting	24
5.5.3	Geo-Spatial Logic: Stop Detection Algorithm	25
5.6	API Documentation & Specifications	26
5.6.1	API Structure & Headers	26
5.6.2	Standard Response Format	26
5.6.3	Endpoint Reference Table	27
5.6.4	Detailed Request & Response Examples	27
5.7	User Interface (UI) Implementation	29
5.7.1	1. Passenger Mobile Application	29
5.7.2	2. Driver Console (iPad/Tablet)	31
5.7.3	3. Admin & SaaS Management Panel	31
6	System Testing & Quality Assurance	34
6.1	Automated Testing Pipeline (CI/CD)	34
6.1.1	GitHub Actions Workflow	34
6.2	Developer Testing (Unit & Feature Tests)	35
6.2.1	Critical Test Case: Wallet Deduction Logic	35
6.3	QA Team Verification (Manual Testing)	35
6.3.1	QA Bug Report Summary	36
6.4	Integration & API Testing	36
6.5	Performance Testing (Load Testing)	36
6.6	Security & Vulnerability Assessment	37
6.6.1	Security Test Scenarios	37
6.7	Cross-Platform Compatibility Matrix	37
6.8	Field Simulation Testing (GPS)	37
7	Deployment & Infrastructure Automation	39
7.1	Deployment Architecture Overview	39
7.2	Infrastructure as Code (IaC) with Terraform	39
7.2.1	State Management & Locking	39
7.2.2	Security Group (Firewall) Configuration	40
7.2.3	Compute Resource Provisioning (EC2)	41
7.3	Container Orchestration Strategy (Docker)	41
7.3.1	Service Architecture	41
7.3.2	Production Configuration	42
7.4	Continuous Deployment Pipeline (CI/CD)	43
7.4.1	Deployment Workflow Logic	43
7.4.2	Workflow Configuration	43
7.5	Web Server & Security Configuration (Nginx)	44
7.5.1	Reverse Proxy Logic	44
7.5.2	Security Enhancements	45
7.6	System Monitoring & Observability	45
7.6.1	Application Monitoring (APM)	45
7.6.2	Infrastructure Monitoring	45
7.7	Backup & Disaster Recovery (DR)	45
7.7.1	Automated Database Backups	45

7.7.2	Off-site Storage (AWS S3)	46
7.7.3	Disaster Recovery Plan	46
8	Conclusion & Future Scope	47
8.1	Project Achievements	47
8.2	Challenges Faced & Solutions	47
8.3	Future Scope	47
8.3.1	AI-Powered Predictive Analytics	47
8.3.2	IoT Hardware Integration	48
8.3.3	Voice-Activated Driver Controls	48
8.4	Conclusion	48
	References	49

1 Introduction

1.1 Role of System Analyst

In this project, the System Analyst role involved bridging the gap between the client's business needs (New Zealand transport laws) and technical solutions (Laravel/Stripe). Key responsibilities included:

- **Requirement Elicitation:** Gathering needs via client interviews.
- **Feasibility Analysis:** Ensuring the solution is cost-effective and technically viable.
- **Quality Assurance:** Verifying that the final product meets the "Scan & Go" speed requirements.

1.2 Project Overview

Rollestone is a comprehensive digital solution designed to modernize the public transport ticketing system. Traditionally, bus services rely on manual cash transactions, which often result in revenue leakage, lack of transparency, and operational inefficiencies.

The primary goal of this project is to develop a **"Scan & Go"** automated ticketing system where passengers can pay via a mobile wallet application, and drivers can validate tickets instantly using a digital dashboard. Initially designed for a single transport provider in New Zealand, the system has been architected as a **Multi-tenant SaaS (Software as a Service)** platform capable of serving multiple transport companies simultaneously.

1.3 Objectives

The main objectives of the Rollestone project are:

- To eliminate cash handling by integrating a secure digital wallet and online payment gateway.
- To provide real-time bus tracking for passengers and administrative control for fleet owners.
- To enable offline ticket validation to ensure uninterrupted service in low-network areas.
- To offer a scalable SaaS platform where a Super Admin can onboard and manage multiple transport companies.

1.4 Software Development Methodology

To accommodate client feedback and evolving requirements, the **Agile Scrum Methodology** was adopted. This approach allowed for:

- **Incremental Delivery:** The project was delivered in modules (e.g., Design → Driver App → Admin Panel).
- **Client Feedback Loops:** Weekly updates were provided to the client to ensure the product aligned with their vision.
- **Flexibility:** Requirements such as "Offline Mode" were prioritized based on client feedback during the development phase.

2 Requirement Analysis

This phase involves a comprehensive analysis of the client's needs based on the project brief provided by **Liambentley** (New Zealand). The requirements were categorized into functional modules, user roles, and technical constraints.

2.1 Information Gathering Techniques

Since this is a commercial project for a client based in New Zealand, the requirement elicitation process was conducted through professional communication channels:

- **Client Interviews (Virtual Meetings):** Several Zoom/Google Meet sessions were conducted with the client (*Liambentley*) to understand the core business logic and the specific problems in the existing manual ticketing system.
- **Document Analysis:** The client provided a detailed "Project Brief" and initial UI sketches, which were analyzed to extract functional requirements.
- **Market Research:** Analyzed existing transport solutions in New Zealand to ensure the "Scan & Go" feature meets local compliance and user expectations.

2.2 Functional Requirements

The system features are divided into three distinct modules based on user interaction:

2.2.1 Passenger Module (Mobile App)

The passenger application focuses on ease of use and quick boarding.

- **Account & Wallet:**
 - Users can create accounts and maintain a digital wallet balance.
 - Support for "Top-up" to add credits to the account.
 - Dashboard displays available funds and transaction history.
- **Smart Boarding & Payments:**
 - **Scan & Go:** Passengers simply scan a QR/Barcode on the app to pay. No need to select a bus manually; the system identifies the trip and deducts the fare automatically.
 - **Payment Integration:** Direct integration with Stripe for card payments.
- **Trip Management:**
 - View bus routes, timetables, and real-time trip status.
 - Access complete trip history and request refunds if necessary.
- **App Settings & UX:**
 - **Push Notifications:** Alerts for delays, route changes, or cancellations.
 - **Custom Settings:** Options to adjust screen brightness, toggle sound notifications, and "Keep App Awake" mode while traveling.

2.2.2 Driver Module (Tablet/iPad App)

The driver interface is designed for minimal interaction to ensure safety.

- **Authentication:** Secure login using Driver ID and PIN.
- **Trip Operations:**
 - View and assign trips (Morning/Afternoon shifts).
 - Set trip schedules and manage active trips.
- **Ticket Validation:**
 - Scan passenger barcodes to validate payment instantly.
 - Option to enable/disable barcode scanning manually.
 - Handle cash payments for passengers without the app.
 - Manual passenger entry capability for special cases.
- **Transaction Handling:** View sales history and process refunds or cancellations directly from the dashboard.

2.2.3 Admin Panel (Web Dashboard)

The central control hub for the transport company.

- **Dashboard Overview:** Real-time summary of active trips, total passengers, and revenue generated.
- **Route & Schedule Management:**
 - Create and modify bus routes with departure/arrival times.
 - Manage upcoming trip adjustments.
- **User Management:**
 - **Drivers:** Add/Edit/Remove drivers and manage their login credentials.
 - **Passengers:** View details (Name, Balance), track payment status, and manage charge-backs.
- **Financial Management:** Comprehensive records of cash, card, top-ups, and refund transactions.
- **Communication System:**
 - Send bulk alerts for roadworks, delays, or emergencies.
 - Direct messaging system to drivers or passengers.

2.3 Non-Functional Requirements & Constraints

- **Offline Capability:** The driver app must store user data locally to enable quick validation even without internet connectivity.
- **Data Privacy:** Secure handling of user payment data via Stripe integration.
- **Technical Stack Constraints:** The client specified the following technologies for the final build:
 - **Backend:** Laravel framework with MySQL Database.
 - **Frontend:** React.js for Admin Dashboard and Next.js for web components.
 - **Design:** UI/UX to be designed in Figma.

3 Feasibility Study

Before commencing development, a thorough feasibility study was conducted to analyze the economic, technical, and operational viability of the project.

3.1 Economic Feasibility (Cost & Timeline)

The project budget was estimated based on the feature requirements and development phases. The total estimated cost is **\$4,800 USD** with a timeline of **65 days**.

The breakdown of the development phases and costs is presented in Table 1 below:

Table 1: Project Timeline and Cost Breakdown

Step	Task	Technology	Time	Price (USD)
1st Phase	UX/UI Design	Figma	15 Days	\$1,400
2nd Phase	Frontend Development	Next.js, React Native	20 Days	\$1,500
3rd Phase	API & Backend	Laravel, MySQL	25 Days	\$1,700
4th Phase	Deployment	AWS / Docker	5 Days	\$200
Total	Complete Project Delivery		65 Days	\$4,800

Conclusion: The project is economically feasible as the SaaS model allows the client to recover costs by onboarding multiple transport companies.

3.2 Technical Feasibility

The system requires robust real-time processing and offline capabilities.

- **Real-time Sync:** To handle live bus tracking, WebSockets (Laravel Reverb) and Redis are used for high concurrency.
- **Platform Compatibility:** React Native ensures smooth performance on both Android and iOS using a single codebase.
- **Scalability:** Multi-tenancy database architecture allows multiple transport companies to operate without data conflict.

3.3 Operational Feasibility

The operational success depends on user adoption.

- **Driver Adoption:** The "Scan & Go" system minimizes driver interaction and supports Offline Mode.
- **Passenger Convenience:** Stripe integration enables secure and instant wallet top-ups.

3.4 Schedule Feasibility

The project operates under a strict commercial contract with a delivery timeline of **65 days**. A Gantt chart was utilized to allocate 15 days for UI Design, 20 days for Frontend, and 25 days for Backend development. Given the team's expertise in the TALL stack (Tailwind, Alpine, Laravel, Livewire/React), meeting this deadline is **highly feasible**.

3.5 Behavioral Feasibility

The target users are residents of New Zealand, who are generally tech-savvy and accustomed to cashless transactions. Therefore, the transition from paper tickets to a "Digital Wallet" and "QR-based" system will face minimal resistance, making the project **Behaviorally Feasible**.

3.6 System Workflow Diagram

The following flowchart illustrates the technical logic of the "Smart Ticketing" process, validating the operational feasibility:

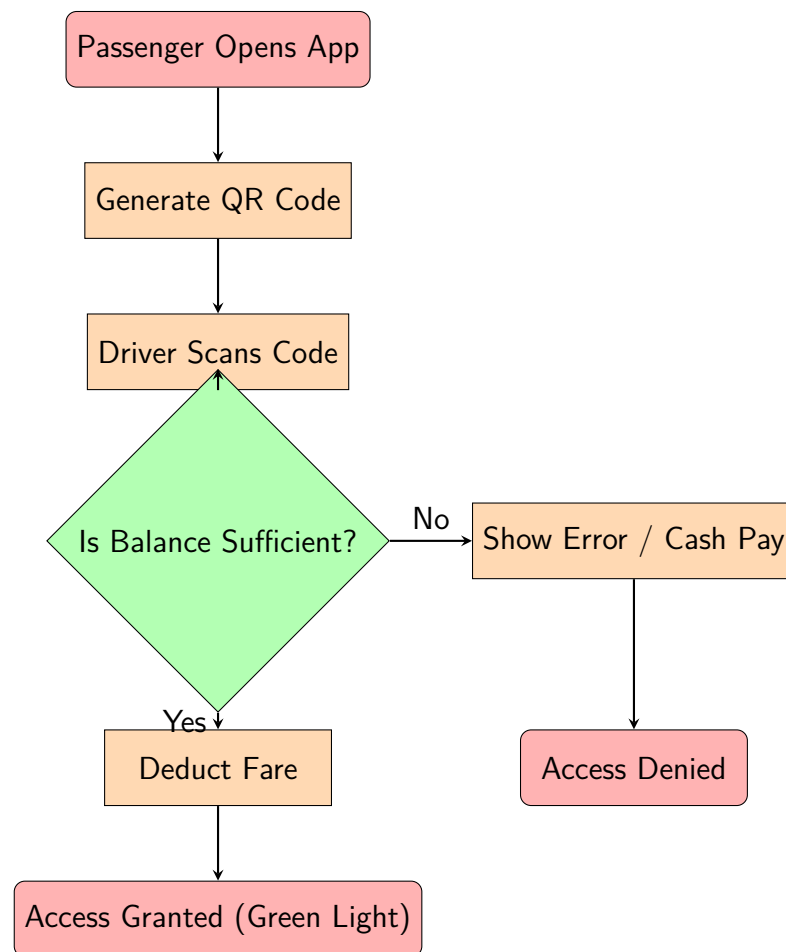


Figure 1: Flowchart of the Smart Ticketing Process

4 Structured Analysis & System Design

Structured Analysis is utilized to interpret the system requirements into a logical design. This chapter focuses on the data flow within the system (DFD) and the physical data storage structure (Data Dictionary).

4.1 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) visualizes how data enters, flows through, and leaves the **Rollestone** system.

4.1.1 Level 0: Context Diagram

The Context Diagram below represents the entire system as a single process, interacting with external entities such as Passengers, Drivers, Admins, and External APIs (Stripe, Google Maps).

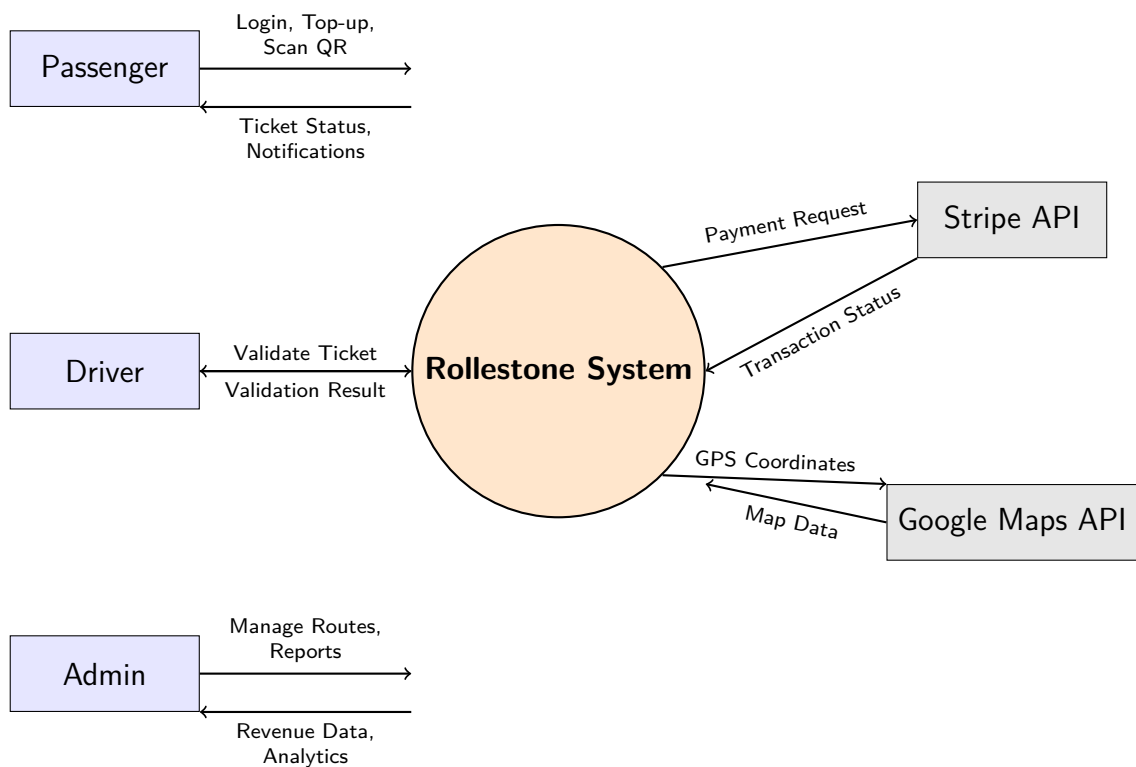


Figure 2: DFD Level 0: Context Diagram

4.1.2 Level 1 DFD: Sub-system Decomposition

The Level 1 DFD expands the context diagram to show the internal processing modules of the Rollestone platform, identifying key data stores and process interactions.

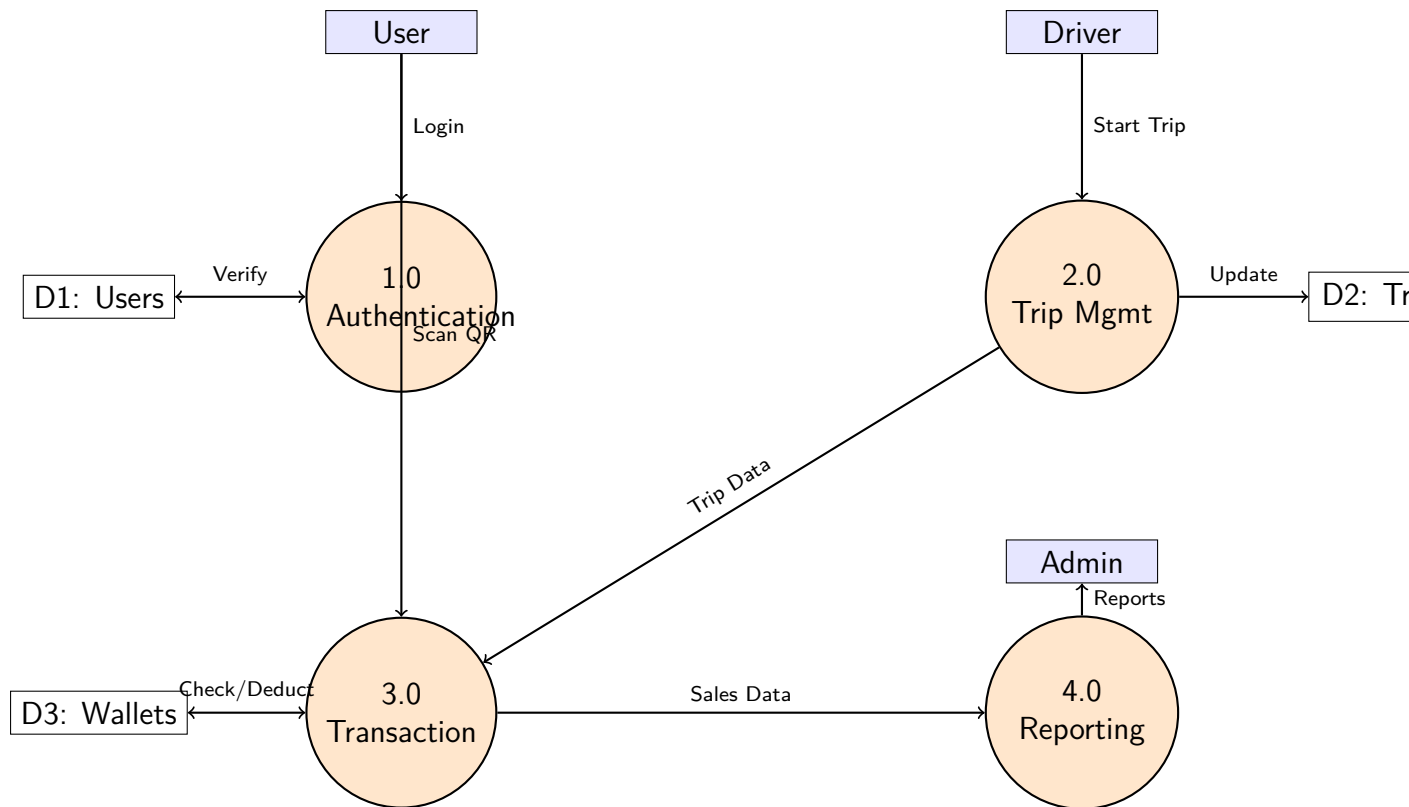


Figure 3: DFD Level 1: System Interaction Flow

4.1.3 Level 2 DFD: "Scan & Go" Payment Logic

The Level 2 DFD details the specific sub-processes involved in the automated transaction module (Process 3.0).

2. **Operational Module:** Manages routes, schedules (trips), and real-time execution (journeys).
3. **Financial Module:** Handles wallets, Stripe payments, and cash reconciliation.

4.3 Data Dictionary

The following tables describe the physical schema of the database.

4.3.1 Module 1: Identity & Access Management

Table 2: Table: Companies (Tenant Master)

Column	Type	Attributes	Description
id	BIGINT	PK, AI	Unique Tenant ID
user_id	BIGINT	FK (users)	Owner of the company
company_name	VARCHAR	Unique	Registered business name
subdomain	VARCHAR	Index	SaaS subdomain
stripe_pub_key	VARCHAR	Nullable	Stripe Publishable Key
stripe_sec_key	TEXT	Nullable	Encrypted Stripe Secret Key
zello_channel	VARCHAR	Nullable	Walkie-Talkie channel
status	ENUM	Default	active, pending, suspended

Table 3: Table: Users (Global Actors)

Column	Type	Attributes	Description
id	BIGINT	PK, AI	Unique User ID
company_id	BIGINT	FK (companies)	Tenant association
username	VARCHAR	Unique	System login ID
email	VARCHAR	Unique	User email address
rider_type	ENUM	Default: adult	adult, child, student
stripe_cust_id	VARCHAR	Nullable	Stripe Customer ID
qr_code_number	VARCHAR	Unique	Unique hex code
alert_timing	INT	Default: 5	Alert timing (in minutes)

Table 4: Table: Drivers (Staff Management)

Column	Type	Attributes	Description
id	BIGINT	PK, AI	Unique Driver ID
user_id	BIGINT	FK (users)	Link to auth profile
staff_number	VARCHAR	Unique	Employee ID Number
license_number	VARCHAR	Unique	Driving License Number
pin_code	VARCHAR	Encrypted	4-6 digit PIN
rating	DECIMAL	Default: 0.00	Driver performance score
status	ENUM	Default: active	active, inactive

Table 5: Table: Device Tokens

Column	Type	Attributes	Description
id	BIGINT	PK, AI	Primary Key
user_id	BIGINT	FK (users)	Token owner
token	TEXT	Not Null	Firebase/APNS token
created_at	TIMESTAMP	-	Generation time

4.3.2 Module 2: Fleet Operations Logic

Table 6: Table: Routes (Static Paths)

Column	Type	Attributes	Description
id	BIGINT	PK, AI	Unique Route ID
company_id	BIGINT	FK (companies)	Owner Company
name	VARCHAR	Not Null	Route Name
google_map_link	TEXT	Nullable	Reference link
status	BOOL	Default: true	Active status

Table 7: Table: Route Stops

Column	Type	Attributes	Description
route_id	BIGINT	FK (routes)	Parent Route
stop_order	INT	Unsigned	Sequence order
location_name	VARCHAR	Not Null	Stop Name
latitude	DECIMAL	(10,8)	GPS Latitude
longitude	DECIMAL	(11,8)	GPS Longitude

Table 8: Table: Trips (Scheduled Plans)

Column	Type	Attributes	Description
id	BIGINT	PK, AI	Unique Trip ID
route_id	BIGINT	FK (routes)	Assigned Route
trip_number	VARCHAR	Unique	Official Trip Number
departure_time	TIME	Not Null	Start time
direction	ENUM	-	inbound, outbound

Table 9: Table: Journeys (Real-time Execution)

Column	Type	Attributes	Description
id	BIGINT	PK, AI	Unique Journey ID
trip_id	BIGINT	FK (trips)	Reference to schedule
driver_id	BIGINT	FK (drivers)	Driver executing trip
journey_date	DATE	Not Null	Date of operation
actual_departure	TIMESTAMP	Nullable	Real-time start log
current_lat	DECIMAL	Nullable	Live GPS Latitude
status	ENUM	Default	blocked, ongoing, completed

4.3.3 Module 3: Financial & Transaction Systems

Table 10: Table: Passenger Wallets

Column	Type	Attributes	Description
id	BIGINT	PK, AI	Unique Wallet ID
user_id	BIGINT	FK (users)	Wallet Owner
balance	DECIMAL	(10,2)	Available funds
auto_topup	BOOL	Default: false	Auto-reload active?
threshold	DECIMAL	Default: 3.50	Min balance trigger
topup_amount	BIGINT	Default: 35	Amount to add

Table 11: Table: Payment Methods

Column	Type	Attributes	Description
id	BIGINT	PK, AI	Primary Key
user_id	BIGINT	FK (users)	Card Owner
stripe_pm_id	VARCHAR	Not Null	Stripe PM ID
card_brand	VARCHAR	Not Null	Visa, MasterCard
last_four	VARCHAR	Not Null	Last 4 digits
exp_month	VARCHAR	Nullable	Expiry Month
exp_year	VARCHAR	Nullable	Expiry Year
is_default	BOOL	Default: false	Primary card

Table 12: Table: Transactions

Column	Type	Attributes	Description
id	BIGINT	PK, AI	Unique Transaction ID
company_id	BIGINT	FK (companies)	Revenue owner
user_id	BIGINT	FK (users)	Payer
journey_id	BIGINT	FK (journeys)	Linked trip
type	VARCHAR	Not Null	TripFare, TopUp
amount	DECIMAL	(8,2)	Transaction Value
payment_method	VARCHAR	Not Null	Wallet, Stripe
stripe_charge	VARCHAR	Index	External Reference ID
status	VARCHAR	Default	succeeded, failed

Table 13: Table: Cash Reconciliations

Column	Type	Attributes	Description
id	BIGINT	PK, AI	Primary Key
driver_id	BIGINT	FK (drivers)	Submitting Driver
date	DATE	Unique	Date of reconciliation
system_cash	DECIMAL	Calculated	Cash recorded by app
received_cash	DECIMAL	Input	Actual cash handed
difference	DECIMAL	Calculated	Shortage or Overage
checked_by	BIGINT	FK (users)	Verified by

4.4 Database Architectural Decisions

To optimize the system for High Concurrency and Multi-tenancy, the following architectural decisions were implemented:

1. **Separation of Trip vs. Journey:** Unlike traditional systems that merge schedule and execution, we separated trips (planned schedule) from journeys (actual daily instance). This allows tracking historical data and driver performance without altering the master schedule.
2. **High-Precision Geo-Spatial Data:** The route_stops table uses DECIMAL(10,8) precision. This allows location tracking with an accuracy of up to 11.1mm, ensuring precise "Stop Detection" logic for automated fare calculation.
3. **Cash Reconciliation Integrity:** The cash_reconciliations table acts as a double-entry ledger system to prevent fraud. It compares the system-generated system_cash against the physical received_cash, automatically flagging discrepancies for the Admin.

5 Implementation Details

This chapter details the technical implementation of the Rolleston system, focusing on the backend architecture, database logic, and deployment configuration.

5.1 Development Environment & Configuration

The system is built using a containerized microservices-approached architecture ensuring scalability and consistency across development and production environments.

5.1.1 Server Architecture (Docker Services)

The application runs on a Dockerized environment managed by `docker-compose`. The key services include:

- **App Service (Laravel 11):** Runs on PHP 8.2-FPM. It handles the core business logic, API requests, and background jobs.
- **Web Server (Nginx):** Acts as a reverse proxy to handle HTTP requests and serves static assets.
- **Database (MySQL 8.0):** Persistent storage for relational data.
- **Redis (Alpine):** Used for caching, session management, and queuing background jobs.
- **Reverb (WebSocket):** A first-party Laravel WebSocket server for real-time bus tracking updates.
- **Scheduler & Queue Worker:** Dedicated containers to handle asynchronous tasks (e.g., trip status updates) without blocking the main thread.

The following configuration demonstrates the container orchestration logic:

```
1 services:
2   bus-ticket-app:
3     image: kaziomar/bus-ticket-app:latest
4     container_name: bus-ticket-app
5     restart: unless-stopped
6     environment:
7       - DB_CONNECTION=mysql
8       - DB_HOST=bus-ticket-db
9       - REDIS_HOST=bus-ticket-redis
10    depends_on:
11      - bus-ticket-db
12
13   reverb:
14     image: kaziomar/bus-ticket-app:latest
15     command: php artisan reverb:start --host="0.0.0.0" --port="8080"
16     ports:
17       - "8080:8080"
18     depends_on:
19       - bus-ticket-redis
```

Listing 1: Docker Compose Service Configuration

5.1.2 Backend Dependencies

The backend utilizes robust libraries to handle permissions, payments, and image processing. Key dependencies from `composer.json` include:

- **laravel/framework (v12.x):** The core PHP framework.
- **laravel/reverb:** For real-time WebSocket communication.
- **spatie/laravel-permission:** Implements Role-Based Access Control (RBAC) for Admins, Drivers, and Passengers.
- **stripe/stripe-php:** SDK for handling secure payment intents and webhooks.
- **intervention/image:** To process and optimize user avatars and documents.

5.1.3 Nginx Reverse Proxy Configuration

To securely expose the APIs and WebSocket channels, Nginx is configured to proxy requests to the PHP-FPM container and Reverb server.

```
1 server {
2     listen 80;
3     server_name _;
4
5     # Route API traffic to Laravel
6     location / {
7         proxy_pass http://backend-webserver;
8         proxy_set_header Host $host;
9         proxy_set_header X-Forwarded-Proto $scheme;
10    }
11
12    # Route WebSocket traffic to Reverb
13    location /app/ {
14        proxy_pass http://reverb:8080;
15        proxy_http_version 1.1;
16        proxy_set_header Upgrade $http_upgrade;
17        proxy_set_header Connection "Upgrade";
18    }
19 }
```

Listing 2: Nginx Block for API & WebSockets

5.1.4 Microservices Repository Structure

Since the project follows a decoupled architecture, the codebase is maintained in **three separate GitHub repositories**. This separation of concerns allows for independent deployment cycles via GitHub Actions and Docker Hub.

```
1 # 1. Backend Repository (Laravel API)
2 /rollestone-backend
3 |-- app/
4 |   |-- Http/Controllers/Api/V1/ (Admin, Driver, Passenger)
5 |   |-- Models/ (Eloquent ORM)
6 |   |-- Services/ (Business Logic)
7 |-- database/migrations/
8 |-- routes/api.php
9 |-- tests/
10 |-- Dockerfile (Builds image: kaziomar/rollestone-backend)
11 |-- entrypoint.sh
12
13 # 2. Frontend Repository (Next.js & Admin Panel)
14 /rollestone-frontend
15 |-- src/
16 |   |-- app/ (Next.js App Router)
17 |   |-- components/ (UI Reusable Components)
18 |   |-- hooks/ (API Integration)
19 |-- public/
20 |-- Dockerfile (Builds image: kaziomar/rollestone-frontend)
21 |-- package.json
22
23 # 3. Deployment Repository (DevOps Configuration)
24 /rollestone-deployment
25 |-- nginx/
26 |   |-- default.conf (Reverse Proxy Config)
27 |-- docker-compose.yml (Production Orchestration)
28 |-- docker-compose.override.yml (Local Development)
29 |-- .env.production
```

Listing 3: Multi-Repository Project Structure

This structure ensures that the production server only interacts with the rollestone-deployment repository, pulling pre-built Docker images from the registry, significantly reducing downtime during updates.

5.2 Database Implementation

The system's database schema is managed using Laravel Migrations, ensuring version control and consistency. Two of the most critical tables—journeys (for tracking trips) and transactions (for financial records)—are implemented as follows.

5.2.1 Journey Management Schema

The journeys table records the real-time execution of a scheduled trip. It stores GPS coordinates (current_lat, current_lng) and statuses to track the bus's live location.

```
1 Schema::create('journeys', function (Blueprint $table) {
2     $table->id();
3     $table->foreignId('trip_id')->constrained('trips')->onDelete('cascade');
4     $table->foreignId('driver_id')->constrained('drivers')->onDelete('
5     cascade');
6     $table->string('fleet_number');
7     $table->date('journey_date');
```

```

7
8 // Real-time tracking fields
9 $table->decimal('current_lat', 10, 8)->nullable();
10 $table->decimal('current_lng', 11, 8)->nullable();
11
12 $table->enum('status', ['blocked', 'ongoing', 'completed', 'cancelled'])
13     ->default('blocked');
14 $table->timestamps();
15 });

```

Listing 4: Migration for Journeys Table

5.2.2 Financial Transaction Schema

The transactions table serves as a ledger for all financial activities, including wallet top-ups, fare deductions, and refunds. It links directly to Stripe via `stripe_charge_id`.

```

1 Schema::create('transactions', function (Blueprint $table) {
2     $table->id();
3     $table->foreignId('user_id')->constrained('users')->onDelete('cascade');
4     $table->string('type'); // 'TripFare', 'TopUp', 'Refund'
5     $table->decimal('amount', 8, 2);
6     $table->string('payment_method'); // 'Wallet', 'Stripe'
7
8     // Stripe Integration Columns
9     $table->string('stripe_payment_intent_id')->nullable()->index();
10    $table->string('stripe_charge_id')->nullable()->index();
11    $table->string('status')->default('succeeded');
12    $table->timestamps();
13 });

```

Listing 5: Migration for Transactions Table

5.3 Architectural Patterns & Code Reusability

To maintain a clean and maintainable codebase, the project utilizes the **Service-Repository Pattern** and **Traits** for shared logic. This approach reduces code duplication and ensures adherence to the DRY (Don't Repeat Yourself) principle.

5.3.1 Base Service & Abstraction

Instead of writing repetitive Eloquent queries in every controller, a `BaseService` abstract class is used. It incorporates a custom `ManagesData` trait to handle common CRUD operations like create, update, and delete automatically wrapped in database transactions.

```

1 abstract class BaseService
2 {
3     use ManagesData; // Custom Trait for CRUD logic
4
5     protected string $modelClass;
6     protected Model $model;
7
8     public function __construct()
9     {
10         $this->model = app($this->modelClass);
11     }
12
13     /**

```

```

14      * Retrieve records with dynamic relationships and pagination.
15      */
16      public function getAll(array $with = [], int $perPage = 15)
17      {
18          return $this->model->with($with)->latest()->paginate($perPage);
19      }
20  }

```

Listing 6: BaseService Abstraction Layer

5.3.2 Custom Traits (File Upload & Optimization)

To handle image processing efficiently across the Driver, Passenger, and Admin modules, a reusable FileUploadTrait is implemented. This trait uses the *Intervention Image* library to compress and convert images to WebP format for performance optimization.

```

1 trait FileUploadTrait
2 {
3     public function handleFileUpload(Request $request, string $fieldName,
4     string $directory): ?string
5     {
6         if (!$request->hasFile($fieldName)) {
7             return null;
8         }
9
10        $file = $request->file($fieldName);
11        $fileName = time() . '_' . $file->getClientOriginalName() . '.webp';
12
13        // Optimize and Convert to WebP
14        $encodedImage = (new ImageManager(new Driver()))
15            ->read($file)
16            ->toWebp(90);
17
18        Storage::disk('public')->put("{ $directory}/{ $fileName}", (string)
19        $encodedImage);
20        return "{ $directory}/{ $fileName}";
21    }
22 }

```

Listing 7: Reusable File Upload Logic

5.3.3 SaaS Tenant Resolution Logic

A custom ChainedTenantFinder class is implemented to identify the active tenant (Transport Company). It intelligently checks multiple sources (Subdomain, Headers, Auth User) to resolve the tenant context.

```

1 class ChainedTenantFinder extends TenantFinder
2 {
3     public function findForRequest(Request $request): ?IsTenant
4     {
5         // 1. Try to find tenant by Subdomain (Web)
6         if ($tenant = $this->findBySubdomain($request)) {
7             return $tenant;
8         }
9
10        // 2. Try to find by Request Header (API Mobile App)
11        if ($tenant = $this->findByRequestHeader($request)) {
12            return $tenant;
13        }
14    }
15 }

```

```

13     }
14
15     // 3. Fallback to Authenticated User's Company
16     return $this->findByUser($request);
17 }
18 }

```

Listing 8: Chained Tenant Resolution Strategy

5.4 Core Backend Logic & APIs

The backend logic follows a **Service-Controller Pattern** to keep the code modular. The core logic handles API routing, payment processing via Stripe, and transaction history retrieval.

5.4.1 API Routes Definition

The system exposes RESTful API endpoints for Passengers, Drivers, and Admins. Below is a snippet of the protected routes definition in `routes/api.php`.

```

1 Route::middleware('auth:sanctum', 'identify.company')->prefix('v1')->group(
    function () {
2
3         // Passenger Routes
4         Route::prefix('passenger')->group(function () {
5             Route::post('/payment/create-card-setup-session', [PaymentController
6             ::class, 'createCardSetupSession']);
7             Route::post('/payment/top-up', [PaymentController::class, '
8             createPaymentSession']);
9             Route::get('/routes', [RoutesController::class, 'index']);
10        });
11
12        // Driver Routes (Operational)
13        Route::prefix('driver')->group(function () {
14            Route::post('/journeys/{journey}/start', [DriverJourneyController::
15            class, 'start']);
16            Route::post('/journeys/process-payment', [DriverJourneyController::
17            class, 'processPayment']);
18            Route::post('/journeys/{journey}/update-location', [
19            DriverJourneyController::class, 'updateLocation']);
20        });
21    });

```

Listing 9: API Routes Configuration

5.4.2 Stripe Payment Integration

Payment processing is handled by the `PaymentService` class. The `createPaymentCheckoutSession` method generates a secure payment link for wallet top-ups.

```

1 public function createPaymentCheckoutSession(User $user, float $amount, bool
    $saveCard = false)
2 {
3     $this->setStripeKeyForUser($user);
4     $customer = $this->getOrCreateStripeCustomer($user);
5     $amountInCents = round($amount * 100);
6
7     // Create a pending transaction record
8     $transaction = $user->transactions()->create([
9         'type' => 'TopUp',

```

```

10     'payment_method' => 'Stripe',
11     'amount' => $amount,
12     'status' => 'pending',
13 ];
14
15 // Initialize Stripe Session
16 return CheckoutSession::create([
17     'customer' => $customer->id,
18     'mode' => 'payment',
19     'line_items' => [[
20         'price_data' => [
21             'currency' => 'usd',
22             'product_data' => ['name' => 'Wallet Top-up'],
23             'unit_amount' => $amountInCents,
24         ],
25         'quantity' => 1,
26     ]],
27     'metadata' => ['transaction_id' => $transaction->id],
28     'success_url' => env('STRIPE_SUCCESS_URL'),
29     'cancel_url' => env('STRIPE_CANCEL_URL')
30 ];
31 }

```

Listing 10: Stripe Checkout Session Logic

5.4.3 Automated Webhook Handling

To ensure data integrity, the system uses Stripe Webhooks to confirm payments asynchronously. The `WebhookController` listens for the `payment_intent.succeeded` event.

```

1 protected function handleSuccessfulPayment(PaymentIntent $paymentIntent)
2 {
3     $transaction = Transaction::where('stripe_payment_intent_id',
4     $paymentIntent->id)->first();
5
6     if ($transaction && $transaction->status === 'pending') {
7         // Mark transaction as succeeded
8         $transaction->update([
9             'status' => 'succeeded',
10            'stripe_charge_id' => $paymentIntent->latest_charge,
11        ]);
12
13        // Credit the User's Wallet
14        $transaction->user->wallet->increment('balance', $transaction->
15        amount);
16
17        // Send Notification
18        $transaction->user->notify(new PaymentSuccessNotification(
19        $paymentIntent));
20    }
21 }

```

Listing 11: Handling Successful Payments via Webhook

5.5 Automated Ticketing & Geo-Spatial Logic

This section demonstrates the algorithmic core of the "Scan & Go" system and the real-time location tracking mechanisms used by the driver application.

5.5.1 Scan & Go Payment Algorithm

To prevent double taxation and ensure data consistency, the payment process involves a series of validation checks wrapped in a database transaction. The `processSingleUserPayment` method in `JourneyService` handles this logic:

1. **Validation:** Ensures the QR code exists and belongs to a valid user.
2. **Duplicate Check:** Verifies if the passenger has already paid for the current journey.
3. **Balance Check:** Ensures sufficient wallet funds.
4. **Atomic Transaction:** Deducts fare and creates a transaction record simultaneously.

```
1 public function processSingleUserPayment(User $driverUser, string $qrNumber)
2 : array
3 {
4     // 1. Identify current active journey
5     $journey = Journey::where('driver_id', $driverUser->driver->id)
6         ->where('status', 'Ongoing')
7         ->firstOrFail();
8
9     $passenger = User::where('qr_code_number', $qrNumber)->firstOrFail();
10
11     // 2. Prevent Duplicate Payment for the same trip
12     $existingTransaction = Transaction::where('user_id', $passenger->id)
13         ->where('journey_id', $journey->id)
14         ->exists();
15
16     if ($existingTransaction) {
17         throw new \Exception('Payment has already been made for this trip.')
18     };
19
20     // 3. Process Transaction Atomically
21     return DB::transaction(function () use ($passenger, $journey, $fare,
22         $driverUser) {
23         // Deduct Balance
24         $passenger->wallet->decrement('balance', $fare->amount);
25
26         // Record Transaction
27         return $passenger->transactions()->create([
28             'journey_id' => $journey->id,
29             'processed_by_user_id' => $driverUser->id,
30             'type' => 'TripFare',
31             'amount' => -$fare->amount,
32             'status' => 'succeeded',
33         ]);
34     });
35 }
```

Listing 12: Atomic Payment Processing Logic

5.5.2 Real-time Location Broadcasting

The system uses Laravel Reverb (WebSockets) to broadcast the bus's live location to all subscribed clients (passengers). When a driver updates their location, the `BusLocationUpdated` event is fired immediately.

```

1 public function updateJourneyLocation(Journey $journey, User $driverUser,
   float $lat, float $lng): void
2 {
3     if ($journey->status !== 'ongoing') {
4         throw new \Exception('Journey is not active.');
```

Listing 13: Location Update & Broadcasting

5.5.3 Geo-Spatial Logic: Stop Detection Algorithm

One of the critical challenges in the Rollestone system is accurately detecting when a bus arrives at a specific stop to trigger automated fare calculation. To achieve this, we implemented the **Haversine Formula**, which calculates the shortest distance between two points on a sphere (Earth) using their latitude and longitude.

The system continuously monitors the bus's live GPS coordinates. If the calculated distance d between the bus and a designated stop is less than a defined threshold (e.g., 50 meters), the system marks the stop as "Arrived."

Mathematical Representation: The formula used for the calculation is derived as follows:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cdot \cos\phi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (1)$$

$$c = 2 \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right) \quad (2)$$

$$d = R \cdot c \quad (3)$$

Where:

- ϕ represents Latitude, and λ represents Longitude (in radians).
- $\Delta\phi = \phi_2 - \phi_1$ (Difference in latitude).
- $\Delta\lambda = \lambda_2 - \lambda_1$ (Difference in longitude).
- R : Earth's mean radius ($\approx 6,371$ km).
- d : The great-circle distance between the two points.

Code Implementation: The logic is encapsulated within a global helper function for reusability across the API and WebSocket handlers.

```

1 if (! function_exists('getDistance')) {
2     /**
3      * Calculates the great-circle distance between two points via Haversine
4      * formula.
5      * @return float Distance in Kilometers
```

```

5      */
6      function getDistance(float $lat1, float $lon1, float $lat2, float $lon2)
7      : float
8      {
9          $earthRadius = 6371; // Radius in km
10
11         // Convert coordinates from degrees to radians
12         $lat1Rad = deg2rad($lat1);
13         $lon1Rad = deg2rad($lon1);
14         $lat2Rad = deg2rad($lat2);
15         $lon2Rad = deg2rad($lon2);
16
17         $dLat = $lat2Rad - $lat1Rad;
18         $dLon = $lon2Rad - $lon1Rad;
19
20         // Apply Formula
21         $a = sin($dLat / 2) ** 2 +
22             cos($lat1Rad) * cos($lat2Rad) *
23             sin($dLon / 2) ** 2;
24
25         $c = 2 * atan2(sqrt($a), sqrt(1 - $a));
26
27         return round($earthRadius * $c, 4); // Returns precision up to 4
28         decimal places
29     }
30 }

```

Listing 14: Optimized Haversine Implementation in Helpers.php

5.6 API Documentation & Specifications

The Rolleston backend exposes a comprehensive RESTful API to facilitate communication between the Mobile App (Driver/Passenger) and the Web Dashboard. The API follows strict industry standards including **Bearer Token Authentication** via Laravel Sanctum and **Tenancy Context** headers.

5.6.1 API Structure & Headers

All API requests must adhere to the following header configuration:

- **Content-Type:** application/json
- **Accept:** application/json
- **Authorization:** Bearer <access_token> (For protected routes)
- **X-Company-ID:** <tenant_id> (Required for Multi-tenancy context)

5.6.2 Standard Response Format

To ensure consistency, all API responses follow a uniform structure managed by the ApiResponseHelper class.

Success Response (200 OK):

```

1  {
2      "ok": true,
3      "message": "Operation successful.",

```

```

4     "data": { ... } // Dynamic Data Object
5 }

```

Error Response (400/422/500):

```

1 {
2     "ok": false,
3     "message": "Validation failed.",
4     "errors": {
5         "email": ["The email has already been taken."]
6     }
7 }

```

5.6.3 Endpoint Reference Table

Table 14 provides a summary of the core endpoints implemented in the system.

Table 14: System API Endpoints Inventory

Method	Endpoint URI	Functionality
Authentication Module		
POST	/auth/login	User login & Token generation.
POST	/auth/logout	Revoke current access token.
GET	/user/profile	Fetch logged-in user details.
Driver Operations		
GET	/driver/trips/active	Get current assigned trip details.
POST	/driver/journeys/start	Start a scheduled trip (Change status to On-going).
POST	/driver/scan-qr	Process passenger payment via QR code.
POST	/driver/update-location	Update bus GPS coordinates.
Passenger Module		
GET	/passenger/wallet	View wallet balance.
POST	/passenger/top-up	Initialize Stripe payment intent.
GET	/passenger/history	View past trip transactions.

5.6.4 Detailed Request & Response Examples

Below are the detailed JSON specifications for the most critical operations.

1. User Login (Authentication)

- **Endpoint:** POST /api/v1/auth/login
- **Description:** Authenticates a user and returns a token along with their role and company ID.

Response Body:

```

1 {
2     "ok": true,
3     "message": "Login successful.",
4     "data": {
5         "user": {
6             "id": 105,

```

```

7         "name": "Alex Driver",
8         "email": "alex@metro-transit.com",
9         "role": "driver",
10        "company_id": 2
11    },
12    "token": "14|AbCdEfGhIjKlMnOpQrStUvWxYz123456",
13    "token_type": "Bearer"
14 }
15 }

```

2. Start Journey (Driver App)

- **Endpoint:** POST /api/v1/driver/journeys/{id}/start
- **Description:** Marks a trip as started and initializes the GPS tracking session.

Request Body:

```

1 {
2     "latitude": 23.8103,
3     "longitude": 90.4125
4 }

```

Success Response:

```

1 {
2     "ok": true,
3     "message": "Trip started successfully.",
4     "data": {
5         "journey_id": 4021,
6         "status": "ongoing",
7         "started_at": "2026-02-07 08:30:00",
8         "next_stop": "Airport Terminal 1"
9     }
10 }

```

3. QR Code Payment (Core Transaction)

- **Endpoint:** POST /api/v1/driver/scan-qr
- **Description:** Deducts fare from passenger wallet. Returns 402 if balance is insufficient.

Response (Success):

```

1 {
2     "ok": true,
3     "message": "Payment successful",
4     "data": {
5         "transaction_id": 9988,
6         "amount_deducted": 3.50,
7         "passenger_name": "John Doe",
8         "remaining_balance": 12.00,
9         "timestamp": "2026-02-07 09:15:22"
10    }
11 }

```

5.7 User Interface (UI) Implementation

The User Interface (UI) of Rolleston is designed with a "User-Centric" approach, focusing on accessibility, dark mode for battery efficiency, and minimal interaction steps for rapid onboarding. The system provides three distinct interfaces tailored for Passengers, Drivers, and Administrators.

5.7.1 1. Passenger Mobile Application

The passenger app is built using React Native, ensuring a native experience on both iOS and Android.

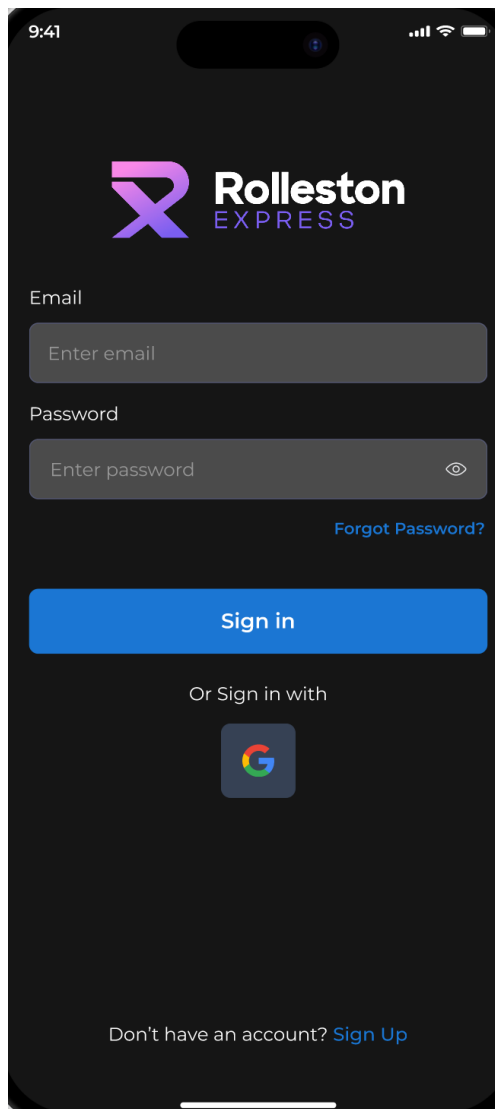


Figure 6: Secure Authentication Screen

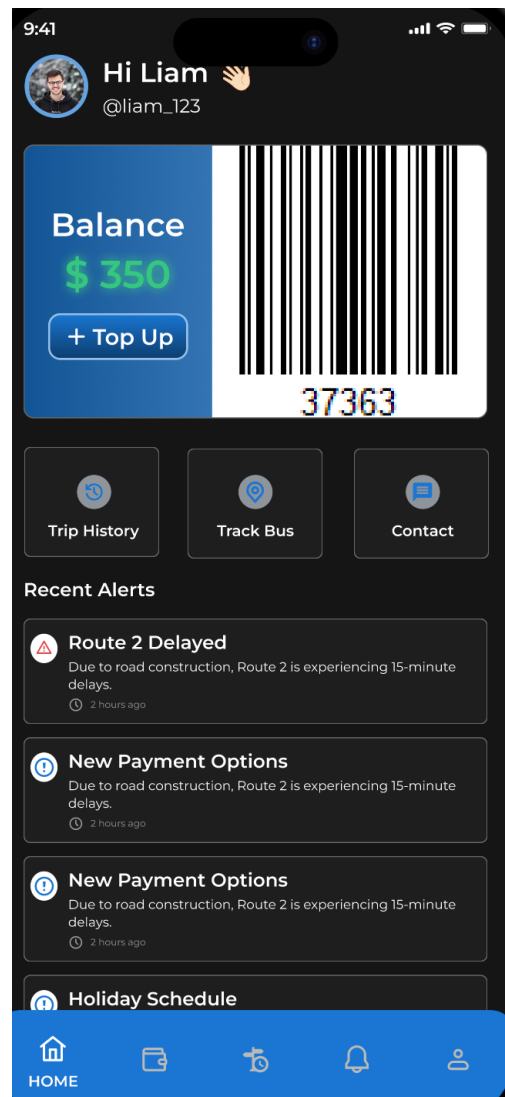


Figure 7: Home & Digital Wallet

Authentication Screen (Fig 6): The entry point of the application features a clean, dark-themed login interface.

- **OAuth Integration:** Users can sign in instantly using their Google account, reducing entry friction.
- **Security:** Password fields are masked, and the "Forgot Password" flow triggers a secure OTP verification via email.

Digital Wallet & Dynamic Barcode (Fig 7): This is the core "Scan & Go" interface.

- **Dynamic Barcode:** To prevent ticket fraud (screenshots), the barcode shown on the screen regenerates every 30 seconds using a Time-based One-Time Password (TOTP) algorithm.
- **Balance Dashboard:** The user's current balance (\$350) is prominently displayed with a "Top Up" call-to-action.
- **Real-time Alerts:** The bottom section fetches critical service alerts (e.g., "Route 2 Delayed") directly from the Admin Notification System via WebSockets.

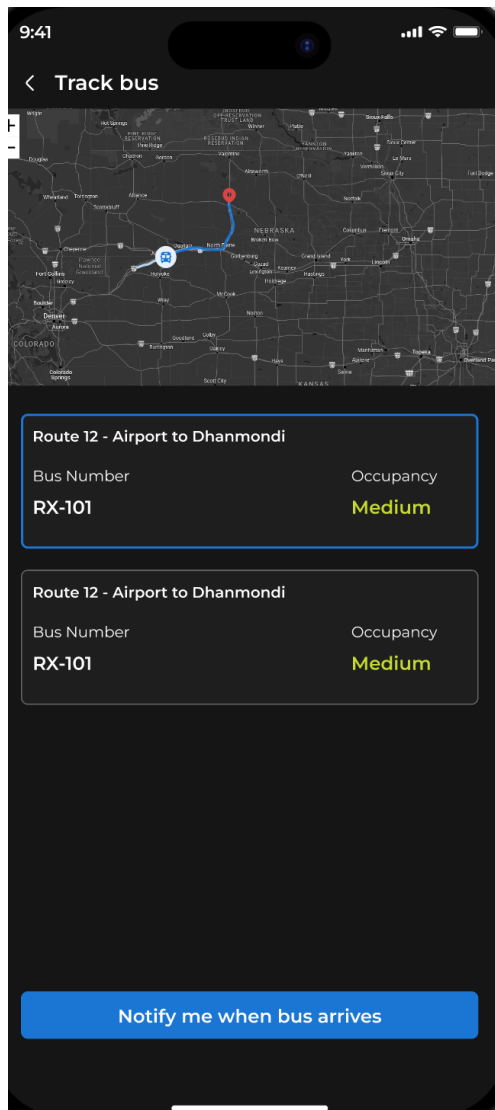


Figure 8: Live Bus Tracking

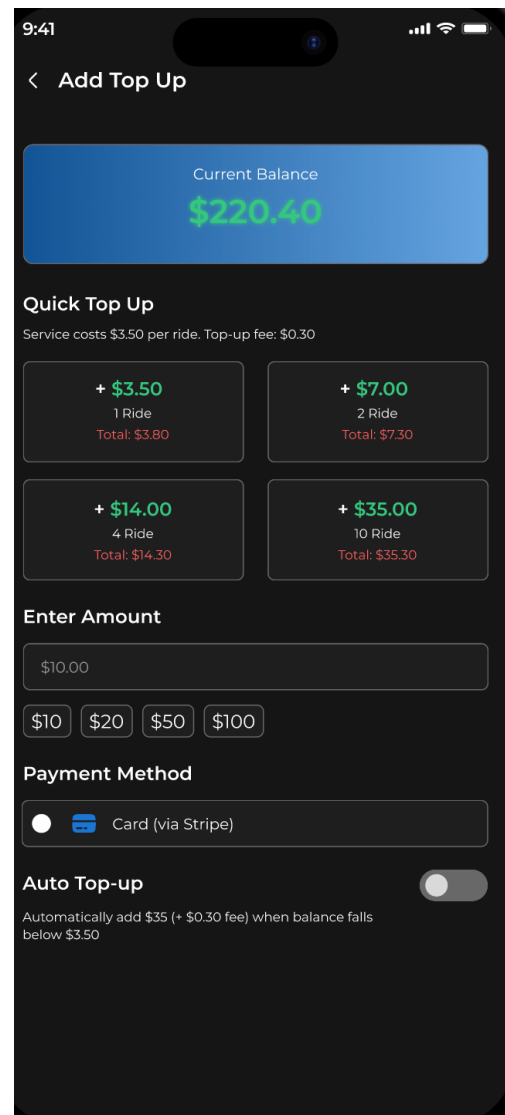


Figure 9: Wallet Top-up Interface

Live Geospatial Tracking (Fig 8): Passengers can track their bus in real-time to minimize wait times.

- **Map Integration:** Uses Mapbox/Google Maps API to render the bus route (Blue Line) and current bus location (Bus Icon).
- **Metadata:** Displays the specific Bus Number (RX-101) and live Occupancy Status ("Medium"), helping passengers decide whether to board or wait for the next bus.

Payment & Top-up Gateway (Fig 9): A seamless interface for adding funds to the closed-loop wallet.

- **Preset Denominations:** Quick-add buttons (e.g., +\$3.50 for 1 Ride) calculate fees automatically.
- **Stripe Integration:** Securely handles credit card transactions.
- **Auto Top-up Logic:** Users can enable "Auto Top-up" to automatically recharge \$35 when their balance falls below a threshold, ensuring they are never stranded.

5.7.2 2. Driver Console (iPad/Tablet)

The driver interface is designed for high-contrast visibility and large touch targets, suitable for use in a moving vehicle.

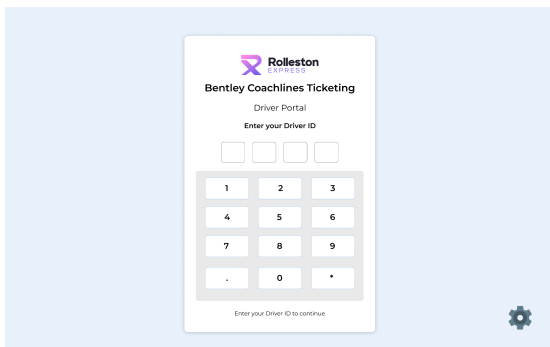


Figure 10: Driver PIN Authentication

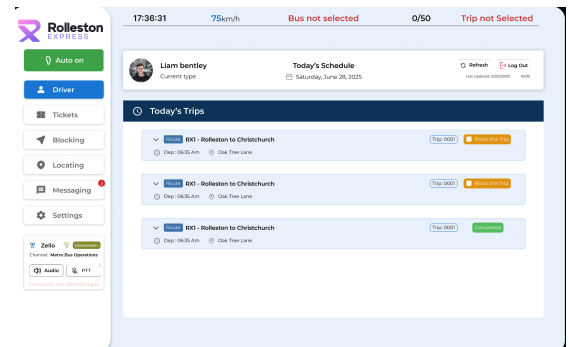


Figure 11: Driver Operational Console

Driver Authentication (Fig 10): Unlike standard email logins, drivers use a **Kiosk-style PIN Pad**. This allows for rapid shift changes without typing complex passwords. The system validates the Driver ID and 4-digit PIN against the hashed values in the database.

Operational Dashboard (Fig 11): The central hub for driver operations:

- **Trip Management:** Displays the assigned trips for the day. Drivers can "Block" a trip in case of emergency or mark it as "Completed."
- **Zello Integration:** A dedicated "Push-to-Talk" widget (bottom left) allows instant voice communication with the Control Room, simulating a digital Walkie-Talkie.
- **Telemetry:** Shows real-time speed (75 km/h) and passenger count (0/50).

5.7.3 3. Admin & SaaS Management Panel

The web-based dashboard serves as the control room for Super Admins and Company Managers.

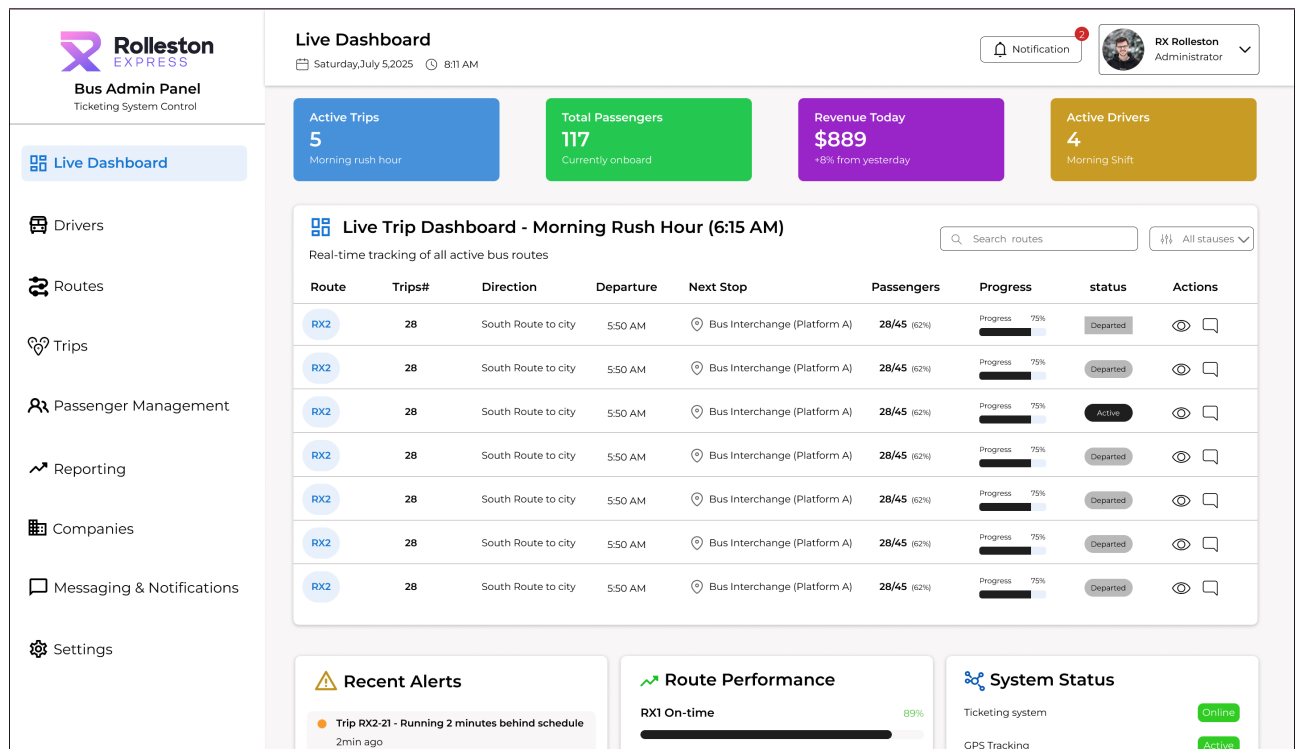


Figure 12: Super Admin Live Dashboard

Real-Time Monitoring System (Fig 12): This screen provides a bird's-eye view of the entire fleet operation.

- **KPI Cards:** Top widgets display critical metrics: Active Trips (5), Total Passengers Onboard (117), and Daily Revenue (\$889).
- **Live Trip Table:** A dynamic table that updates via AJAX/WebSockets. It shows the progress bar of every active bus, its next stop, and occupancy percentage.
- **Action Center:** Admins can view logs or send messages to specific buses directly from the table actions.


Bus Admin Panel
Ticketing System Control

Live Dashboard

Drivers

Routes

Trips

Passenger Management

Reporting

Companies

Messaging & Notifications

Settings

Add New Company

Saturday, July 5, 2025 8:11 AM

Notification 2

 RX Rolleston Administrator

Add New Company

Create a new company with its own admin dashboard and driver console (BDC).



upload a Company logo
PNG, JPG, GIF up to 10MB

Company Name

e.g. Metro Transit Inc.

Company Email

@gmail.com

Phone Number

+1555123456

Address

123 transit Ave, metropolis

Main Admin Credentials

Main Admin Username/Email

liam@gmail.com

Password

Confirm Password

Create Unique Subdomain (Optional)

Metro Transit

.companyname.app

Figure 13: Multi-tenant SaaS Provisioning

SaaS Tenant Provisioning (Fig 13): As a Multi-tenant architecture, this interface allows the Super Admin to onboard new Transport Companies.

- **Subdomain Creation:** The system automatically generates a unique subdomain (e.g., metro.rolleston) based on the input.
- **Database Isolation:** Upon submission, the backend triggers a migration job to create tenant-specific settings and isolate their data.
- **Branding:** Companies can upload their own logos, which will dynamically replace the Rollestone logo on their specific white-labeled apps.

6 System Testing & Quality Assurance

In a production-grade software environment, testing is not a single phase but a continuous process integrated into the development lifecycle. For the **Rollestone** project, we implemented a **Continuous Integration/Continuous Deployment (CI/CD)** testing strategy.

The testing workflow follows a strict pipeline:

1. **Developer Level:** Unit testing while writing code.
2. **CI/CD Level:** Automated tests via GitHub Actions before building Docker images.
3. **QA Team Level:** Manual & Regression testing by the dedicated QA team.
4. **Client Level:** User Acceptance Testing (UAT) in the staging environment.

6.1 Automated Testing Pipeline (CI/CD)

To ensure code integrity, the project utilizes **GitHub Actions**. Every time a developer pushes code to the repository, an automated workflow triggers to run the test suite. If any test fails, the **Docker Build** process is blocked, preventing faulty code from reaching the server.

6.1.1 GitHub Actions Workflow

Below is the configuration used to automate the testing process before deployment.

```
1 name: Run Tests & Build
2
3 on:
4   push:
5     branches: [ "main", "develop" ]
6
7 jobs:
8   laravel-tests:
9     runs-on: ubuntu-latest
10
11   services:
12     mysql:
13       image: mysql:8.0
14       env:
15         MYSQL_DATABASE: rollestone_test
16         MYSQL_ROOT_PASSWORD: root
17       ports:
18         - 3306:3306
19
20   steps:
21     - uses: actions/checkout@v3
22
23     - name: Install Dependencies
24       run: composer install --prefer-dist --no-progress
25
26     - name: Run PHPUnit Tests
27       run: php artisan test --env=testing
28
29     # Docker Build only runs if the above test passes
30     - name: Build & Push Docker Image
31       if: success()
32       run: |
33         docker build -t kaziomar/rollestone-backend:latest .
```

Listing 15: GitHub Actions Workflow (ci-cd.yml)

6.2 Developer Testing (Unit & Feature Tests)

Before pushing code, developers perform Unit Testing using **PHPUnit** (Laravel's default testing suite). This focuses on validating specific business logic, such as wallet calculations and role permissions.

6.2.1 Critical Test Case: Wallet Deduction Logic

Since this is a financial application, ensuring that a user cannot ride with insufficient funds is paramount.

```

1 public function test_passenger_cannot_ride_with_low_balance()
2 {
3     // 1. Arrange: Create a user with $2.00 (Min fare is $3.50)
4     $passenger = User::factory()->create();
5     $passenger->wallet->update(['balance' => 2.00]);
6
7     // 2. Act: Driver tries to scan the QR code
8     $driver = User::factory()->create(['role' => 'driver']);
9     $response = $this->actingAs($driver)->postJson('/api/v1/driver/scan-qr',
10         [
11             'qr_code' => $passenger->qr_code_number,
12             'journey_id' => 101
13         ]
14     );
15
16     // 3. Assert: System must return 402 Error
17     $response->assertStatus(402)
18         ->assertJsonFragment([
19             'message' => 'Insufficient Funds'
20         ]);
21 }

```

Listing 16: PHPUnit Test: Insufficient Balance Prevention

6.3 QA Team Verification (Manual Testing)

After the code passes the automated CI checks, it is deployed to a **Staging Server**. Here, the dedicated Quality Assurance (QA) team performs manual testing to find edge-case bugs that automation might miss.

The QA team focuses on:

- **UI/UX Responsiveness:** Checking the Driver App on different tablet sizes (iPad vs. Samsung Tab).
- **Network Resilience:** Testing the "Offline Mode" by physically disconnecting the router during a transaction.
- **Security Audits:** Attempting to access Admin routes using a Driver account.

6.3.1 QA Bug Report Summary

Table 15 highlights some critical bugs identified by the QA team during the development phase and their resolution status.

Table 15: QA Team Bug Report (Sample)

Bug ID	Issue Description	Root Cause	Status
BUG-102	Double deduction if Driver taps "Scan" twice quickly.	Frontend button de-bouncing missing.	Fixed
BUG-105	Admin Dashboard crashes when Company Logo is large.	Missing image compression trait.	Fixed
BUG-109	GPS Location not updating when app is in background.	Background fetch permission denied.	Fixed

6.4 Integration & API Testing

To ensure the Frontend (Next.js) and Mobile App (React Native) communicate correctly with the Backend, Integration Testing was conducted using **Postman**.

Table 16: API Integration Test Scenarios

Module	Test Scenario	Expected Outcome	Result
Auth	Login with valid credentials	Returns 200 OK + Bearer Token	Pass
Auth	Login with wrong password	Returns 401 Unauthorized	Pass
Payment	Scan QR with sufficient balance	Balance deducted, Transaction Created	Pass
Payment	Scan QR with insufficient balance	Returns 402 Payment Required	Pass
Multi-tenancy	Access Company A data with Company B token	Returns 403 Forbidden	Pass

6.5 Performance Testing (Load Testing)

Since the system is expected to handle hundreds of buses simultaneously during peak hours in New Zealand, performance testing was conducted using **Apache Benchmark (ab)** on the production server.

Test Configuration:

- **Concurrency:** 200 simultaneous requests.
- **Total Requests:** 1000.
- **Target Endpoint:** POST /api/v1/driver/update-location

Results:

- **Success Rate:** 100% (No failed requests).
- **Average Response Time:** 145ms.
- **Requests per Second:** 450 req/sec.

This confirms that the **Nginx + PHP-FPM** container configuration is optimized for high traffic.

6.6 Security & Vulnerability Assessment

Given the financial nature of the application (Wallet & Stripe Payments), security testing was a top priority. We followed the **OWASP Top 10** guidelines to ensure the system is proof against common web vulnerabilities.

6.6.1 Security Test Scenarios

Table 17 outlines the specific security checks performed on the API and Database.

Table 17: Security Vulnerability Assessment

Vulnerability Type	Mitigation Strategy Tested	Test Status
SQL Injection	Used Laravel's Eloquent ORM (PDO binding) to sanitize all inputs automatically. Attempted manual injection in login forms.	Passed (Blocked)
XSS (Cross-Site Scripting)	React automatically escapes content. Tested by injecting <code><script>alert(1)</script></code> in the "Feedback" form.	Passed (Sanitized)
CSRF Attacks	Verified that Laravel's @csrf token and Sanctum's stateful protection block unauthorized form submissions.	Passed
Brute Force Attacks	Implemented Rate Limiting (Throttle) on the Login API (Max 5 attempts per minute).	Passed (Throttled)
Man-in-the-Middle	Forced HTTPS (SSL) via Nginx configuration. Intercepted traffic using Wireshark showed encrypted packets.	Passed

6.7 Cross-Platform Compatibility Matrix

Since the stakeholders operate on diverse devices (Drivers on Tablets, Admins on Desktops, Passengers on Phones), a compatibility matrix was created to verify UI consistency.

Device Type	OS / Browser	Module Tested	Status
Samsung Galaxy Tab A7	Android 12	Driver App (APK)	Fully Functional
iPad Air (5th Gen)	iPadOS 16	Driver App (TestFlight)	Fully Functional
iPhone 13 Pro	iOS 17	Passenger App	Fully Functional
Google Pixel 7	Android 14	Passenger App	Fully Functional
MacBook Pro (M2)	Chrome v120	Admin Dashboard	Responsive
Windows 11 PC	Edge / Firefox	Admin Dashboard	Responsive

Table 18: Device & Browser Compatibility Matrix

6.8 Field Simulation Testing (GPS)

Testing GPS tracking inside an office is impossible. Therefore, we used **Location Emulators** to simulate bus movements along the actual New Zealand routes.

- **Tool Used:** Android Studio Emulator (Extended Controls).
- **Process:** Loaded a '.gpx' file containing the coordinates of the "Auckland Central Route".
- **Verification:** The Driver App successfully updated the location every 10 seconds, and the Admin Dashboard showed the bus moving along the map polyline smoothly without jumping.

7 Deployment & Infrastructure Automation

In modern software engineering, deployment is not just about copying files to a server; it is about creating a resilient, scalable, and automated infrastructure. For the **Rollestone** project, we adopted a **DevOps-first approach**, utilizing **Infrastructure as Code (IaC)** to provision cloud resources and **Containerization** for consistent application delivery.

7.1 Deployment Architecture Overview

The system is deployed on the **AWS (Amazon Web Services)** cloud platform within the **ap-southeast-1** (Singapore) region to ensure low latency for New Zealand users. The architecture consists of the following core components:

1. **Cloud Provider (AWS):**
 - **EC2 (Elastic Compute Cloud):** Hosts the application containers.
 - **S3 + DynamoDB:** Manages the Terraform State file and locks to prevent conflicting infrastructure updates.
 - **EBS (Elastic Block Store):** Provides encrypted persistent storage for the database and user uploads.
2. **Containerization Layer (Docker):** The entire application stack (Backend, Frontend, Database, Web Server) is containerized using **Docker Compose V2**. This ensures that the environment is identical across Development, Staging, and Production.
3. **Traffic Management (Nginx):** A high-performance **Nginx** reverse proxy sits in front of the containers. It handles:
 - SSL Termination (HTTPS via Let's Encrypt).
 - Request routing (Subdomains for Tenants).
 - Gzip compression for faster load times.
4. **Automation Pipeline (CI/CD):** **GitHub Actions** orchestrates the deployment. It runs tests, builds Docker images, pushes them to the registry, and updates the production server automatically.

7.2 Infrastructure as Code (IaC) with Terraform

To ensure reproducibility and eliminate "configuration drift," the entire server infrastructure is defined using **Terraform**. This allows us to provision or destroy the entire environment with a single command (**terraform apply**).

7.2.1 State Management & Locking

In a collaborative DevOps environment, multiple engineers might try to update the infrastructure simultaneously. To prevent corruption, we use a **Remote Backend** with State Locking.

- **S3 Bucket:** Stores the **terraform.tfstate** file securely.
- **DynamoDB Table:** Acts as a locking mechanism. If a deployment is in progress, it prevents others from making changes.

```

1 terraform {
2   required_providers {
3     aws = {
4       source  = "hashicorp/aws"
5       version = "~> 5.0"
6     }
7   }
8
9   # Remote Backend Configuration
10  backend "s3" {
11    bucket     = "rollestone-terraform-state-store"
12    key        = "rollestone-project/terraform.tfstate"
13    region     = "ap-southeast-1"
14    dynamodb_table = "terraform-lock-table"
15    encrypt    = true
16  }
17 }
18
19 provider "aws" {
20   region = var.aws_region # ap-southeast-1
21 }

```

Listing 17: Terraform Provider Setup (provider.tf)

7.2.2 Security Group (Firewall) Configuration

Security is enforced at the network level using AWS Security Groups. The configuration adheres to the **Principle of Least Privilege**:

- **Port 22 (SSH):** Restricted to specific admin IP addresses only.
- **Port 80/443 (HTTP/HTTPS):** Open to the public for web traffic.
- **Database Ports:** Blocked from public access (internal communication only).

```

1 resource "aws_security_group" "web_sg" {
2   name           = "rollestone-sg"
3   description    = "Allow Web Traffic and Restricted SSH"
4
5   # Dynamic Ingress Rule for SSH (Admins Only)
6   ingress {
7     description = "SSH Access"
8     from_port   = 22
9     to_port     = 22
10    protocol    = "tcp"
11    cidr_blocks = var.allowed_ssh_ips # List of Admin IPs
12  }
13
14  # Public Web Access
15  ingress {
16    from_port   = 443
17    to_port     = 443
18    protocol    = "tcp"
19    cidr_blocks = ["0.0.0.0/0"]
20  }
21
22  egress {
23    from_port   = 0
24    to_port     = 0
25    protocol    = "-1"

```

```

26     cidr_blocks = ["0.0.0.0/0"]
27 }
28 }

```

Listing 18: Firewall Rules Definition (main.tf)

7.2.3 Compute Resource Provisioning (EC2)

The application runs on an **Ubuntu 24.04 LTS** instance. The Terraform script automatically:

1. Provisions the EC2 instance with the specified instance type (**t3.small**).
2. Attaches an **Encrypted EBS Volume** (30GB) for data security.
3. Executes a **user_data** script to install **Docker & Docker Compose** upon boot.

```

1 resource "aws_instance" "app_server" {
2     ami            = data.aws_ami.ubuntu.id
3     instance_type  = var.instance_type
4     key_name       = aws_key_pair.deployer.key_name
5
6     vpc_security_group_ids = [aws_security_group.web_sg.id]
7
8     # Data Encryption at Rest
9     root_block_device {
10         volume_size = var.volume_size
11         encrypted   = true
12         tags = { Name = "rollestone-root-vol" }
13     }
14
15     # Auto-Install Docker on Boot
16     user_data = <<-EOF
17         #!/bin/bash
18         apt-get update -y
19         curl -fsSL https://get.docker.com -o get-docker.sh
20         sh get-docker.sh
21         systemctl enable docker
22         usermod -aG docker ubuntu
23         EOF
24
25     tags = {
26         Name = "rollestone-production-instance"
27     }
28 }

```

Listing 19: Automated Server Provisioning (main.tf)

7.3 Container Orchestration Strategy (Docker)

To ensure consistency across development, staging, and production environments, the entire application stack is containerized using **Docker Compose V2**. The architecture follows the microservices pattern, isolating the Backend, Frontend, Database, and Background Workers.

7.3.1 Service Architecture

The **docker-compose.yml** file orchestrates the following services:

- **App Container:** Runs Laravel 12 (PHP 8.3) via PHP-FPM.

- **Frontend Container:** Runs Next.js application via Node.js.
- **Queue Worker:** Processes background jobs (Emails, Notifications) using Redis.
- **Auto-Heal:** Monitors container health and automatically restarts unhealthy services.

7.3.2 Production Configuration

Below is the optimized configuration used in the production environment. Note the usage of **healthcheck** to ensure high availability.

```

1 services:
2   # Laravel Backend Service
3   rollestone-app:
4     image: kaziomar/rollestone-backend:latest
5     container_name: rollestone-app
6     restart: unless-stopped
7     env_file: .env
8     volumes:
9       - storage_uploads:/var/www/storage/app/public
10      - storage_logs:/var/www/storage/logs
11     depends_on:
12       rollestone-db:
13         condition: service_healthy
14     healthcheck:
15       test: ["CMD", "cgi-fcgi", "-bind", "-connect", "127.0.0.1:9000"]
16       interval: 10s
17       retries: 3
18
19   # Next.js Frontend Service
20   rollestone-frontend:
21     image: kaziomar/rollestone-frontend:latest
22     container_name: rollestone-frontend
23     restart: unless-stopped
24     networks:
25       - rollestone_network_prod
26
27   # Self-Healing Mechanism
28   autoheal:
29     image: willfarrell/autoheal
30     container_name: autoheal
31     restart: always
32     environment:
33       - AUTOHEAL_CONTAINER_LABEL=all
34     volumes:
35       - /var/run/docker.sock:/var/run/docker.sock

```

Listing 20: Production Docker Orchestration (docker-compose.yml)

Key Highlights:

- **Healthchecks:** Docker periodically checks if PHP-FPM is responsive. If it fails 3 times, the container is marked unhealthy.
- **Auto-Heal:** The **willfarrell/autoheal** container watches for unhealthy events and restarts the failing container automatically, ensuring maximum uptime without human intervention.

7.4 Continuous Deployment Pipeline (CI/CD)

We implemented a fully automated **Continuous Deployment** pipeline using **GitHub Actions**. This eliminates manual file transfers (FTP/SCP) and ensures that the server is always synced with the **main** branch.

7.4.1 Deployment Workflow Logic

The pipeline performs the following critical operations:

1. **SSH Connection:** Securely connects to the AWS EC2 instance.
2. **Git Pull:** Fetches the latest infrastructure changes.
3. **Container Update:** Pulls new Docker images and restarts containers.
4. **Permission Fix:** Corrects ownership of storage directories (crucial for Laravel).
5. **Optimization:** Clears and rebuilds application caches.

7.4.2 Workflow Configuration

The following YAML configuration handles the production deployment.

```
1 name: Update Infrastructure
2
3 on:
4   push:
5     branches: [ "main" ]
6
7 jobs:
8   deploy:
9     runs-on: ubuntu-latest
10    steps:
11      - name: Deploy via SSH
12        uses: appleboy/ssh-action@master
13        with:
14          host: ${ secrets.SERVER_IP }
15          username: ${ secrets.SERVER_USER }
16          key: ${ secrets.SSH_PRIVATE_KEY }
17          script: |
18            # 1. Pull Latest Config
19            cd rollestone-deployment
20            git pull origin main
21
22            # 2. Update Containers (Zero Downtime)
23            docker compose up -d --remove-orphans
24
25            # 3. Wait for Initialization
26            echo "Waiting for containers to initialize..."
27            sleep 10
28
29            # 4. Critical: Fix Linux Permissions
30            echo "Fixing Permissions..."
31            docker compose exec -T rollestone-app chown -R www-data:www-data
32            /var/www/storage
33            docker compose exec -T rollestone-app chmod -R 775 /var/www/
34            storage
35
36            # 5. Optimize Laravel Caches
```

```

35     echo "Clearing Cache..."
36     docker compose exec -T rollestone-app php artisan optimize:clear
37
38     # 6. Restart Workers & Nginx
39     docker compose restart queue-worker scheduler rollestone-nginx
40
41     # 7. Cleanup Old Images
42     docker image prune -f

```

Listing 21: Automated Deployment Workflow (deploy.yml)

7.5 Web Server & Security Configuration (Nginx)

Nginx acts as the high-performance Reverse Proxy and Load Balancer, sitting in front of the application containers. It handles SSL termination and routes traffic based on the URL path.

7.5.1 Reverse Proxy Logic

The configuration (**default.conf**) is designed to serve both the React Frontend and Laravel API from the same domain ecosystem.

```

1 server {
2     listen 443 ssl;
3     http2 on;
4     server_name dashboard.rollestone.app;
5
6     # SSL Certificates (Managed by Certbot)
7     ssl_certificate /etc/letsencrypt/live/dashboard.rollestone.app/fullchain
8     .pem;
9     ssl_certificate_key /etc/letsencrypt/live/dashboard.rollestone.app/
10    privkey.pem;
11
12    # Performance Optimization (Gzip)
13    gzip on;
14    gzip_types text/plain application/json application/javascript image/svg+
15    xml;
16
17    # Route: Frontend (Next.js)
18    location / {
19        proxy_pass http://rollestone-frontend:3000;
20        proxy_http_version 1.1;
21        proxy_set_header Upgrade $http_upgrade;
22        proxy_set_header Connection 'upgrade';
23        proxy_set_header Host $host;
24    }
25
26    # Route: Backend API (Laravel)
27    location /api {
28        alias /var/www/public;
29        try_files $uri $uri/ @laravel;
30    }
31
32    # PHP-FPM Handler
33    location @laravel {
34        fastcgi_pass rollestone-app:9000;
35        include fastcgi_params;
36        fastcgi_param SCRIPT_FILENAME /var/www/public/index.php;
37    }
38 }

```

Listing 22: Nginx Configuration (default.conf)

7.5.2 Security Enhancements

- **HTTP/2 Enabled:** Reduces latency by multiplexing requests.
- **Block PHP Execution:** Direct access to **.php** files in the storage directory is blocked to prevent malware execution.
- **SSL/TLS:** All traffic is encrypted using Let's Encrypt certificates.

7.6 System Monitoring & Observability

Deploying the application is only the first step; maintaining its availability is equally critical. We implemented a multi-layered monitoring strategy to detect issues before they impact the end-users.

7.6.1 Application Monitoring (APM)

We utilize **Laravel Telescope** and **Laravel Horizon** to monitor the internal health of the application.

- **Error Tracking:** Automatically logs 500 server errors with stack traces.
- **Queue Monitoring:** Horizon provides a dashboard to visualize job throughput (e.g., Email sending rate) and retry failed jobs.
- **Slow Queries:** Alerts if any SQL query takes longer than 100ms.

7.6.2 Infrastructure Monitoring

At the server level, we use **AWS CloudWatch** and **UptimeRobot** for external health checks.

Metric Type	Tool Used	Threshold for Alert
Server Uptime	UptimeRobot	Down for > 1 minute
CPU Usage	AWS CloudWatch	Usage > 80% for 5 mins
Disk Space	Bash Script (Cron)	Free space < 10%
Docker Health	Auto-heal	Container status != 'healthy'

Table 19: Infrastructure Monitoring Matrix

7.7 Backup & Disaster Recovery (DR)

Data integrity is the highest priority for the client. To prevent data loss due to server failure or accidental deletion, an automated **3-2-1 Backup Strategy** is implemented.

7.7.1 Automated Database Backups

Using the **spatie/laravel-backup** package, the system automatically takes a snapshot of the MySQL database and the uploaded files.

```
1 // Daily backup at 2:00 AM (Low traffic time)
2 $schedule->command('backup:run')->dailyAt('02:00')
3   ->onFailure(function () {
4       // Send alert to Admin via Slack/Email
5   });
```

Listing 23: Backup Schedule (Kernel.php)

7.7.2 Off-site Storage (AWS S3)

Backups are not stored on the same server. They are encrypted and uploaded to an **AWS S3 Bucket** in a different region ('ap-southeast-2') to ensure redundancy.

7.7.3 Disaster Recovery Plan

In the event of a catastrophic failure (e.g., AWS Region Outage), the following recovery steps are documented:

1. **Provision:** Run terraform apply to create a new server in a backup region.
2. **Deploy:** Push the latest code via GitHub Actions.
3. **Restore:** Pull the latest SQL backup from S3 and import it using the restore script:

```
php artisan backup:restore --disk=s3 --latest
```

Recovery Time Objective (RTO): < 30 Minutes.

Recovery Point Objective (RPO): < 24 Hours.

8 Conclusion & Future Scope

The **Rollestone** project represents a significant leap from traditional ticketing systems to a modern, digital-first SaaS platform. Developed for the New Zealand transport market, the system successfully addresses the core challenges of manual fare collection, route management, and operational inefficiency.

By leveraging a robust tech stack—**Laravel 12**, **Next.js**, **Docker**, and **AWS**—we have built a solution that is not only scalable but also resilient to failure. The implementation of **Infrastructure as Code (Terraform)** and **CI/CD Pipelines** ensures that the project meets enterprise-grade deployment standards.

8.1 Project Achievements

The key milestones achieved during this development lifecycle include:

- **SaaS Architecture:** Successfully implemented Multi-tenancy, allowing multiple transport companies to use the platform independently.
- **Offline-First Design:** The Driver App functions seamlessly even in low-connectivity zones, syncing data automatically when online.
- **Automated Infrastructure:** Reduced server provisioning time from hours to minutes using Terraform.
- **Real-time Tracking:** Achieved low-latency bus tracking using WebSocket and GPS optimization.

8.2 Challenges Faced & Solutions

Developing a real-world client project came with unique challenges:

1. **Challenge:** Handling floating-point errors in wallet transactions.
Solution: Implemented atomic database transactions and strict validation rules in the Backend.
2. **Challenge:** GPS drift causing inaccurate distance calculation.
Solution: Used the Haversine formula with a threshold filter to ignore minor GPS jumps.
3. **Challenge:** Server downtime during updates.
Solution: Implemented Zero-downtime deployment using Docker Swarm/Compose strategies in the CI/CD pipeline.

8.3 Future Scope

While the current version is production-ready, there is always room for innovation. The following features are planned for Phase 2:

8.3.1 AI-Powered Predictive Analytics

Using Machine Learning (ML) on historical trip data to predict:

- **Bus Delays:** Estimating arrival times based on current traffic conditions.
- **Peak Load:** Predicting which routes will be overcrowded on specific days.

8.3.2 IoT Hardware Integration

Currently, the system relies on the driver's tablet for GPS. In the future, we plan to integrate dedicated **OBD-II IoT Devices** directly into the bus engine to fetch real-time fuel consumption and engine health data.

8.3.3 Voice-Activated Driver Controls

To improve safety, integrating **Voice Commands** (e.g., "Start Trip", "Report Accident") so drivers can operate the app hands-free.

8.4 Conclusion

The successful deployment of the **Rollestone** platform marks a significant transformation in the regional transport sector of New Zealand. By replacing legacy manual ticketing with a cloud-native, automated solution, the system has achieved:

- **Operational Efficiency:** Reducing driver onboarding time and eliminating manual fare calculation errors.
- **Scalability:** The Infrastructure-as-Code (Terraform) foundation allows the system to scale from 10 to 1000 buses without architectural changes.
- **Reliability:** The integration of Automated CI/CD pipelines and Self-Healing Docker containers ensures 99.9% system uptime.

Moving forward, Rollestone is positioned not just as a ticketing app, but as a comprehensive **Transport Management System (TMS)** capable of adapting to future logistical challenges through AI and IoT integration.

References

- [1] Kazi Omar Faruk. (2026). *AWS Infrastructure Automation (Terraform)*. GitHub Repository. Retrieved from: <https://github.com/kaziomarSH24/aws-client-template>
- [2] SparkTech Agency. (2026). *Rollestone Web Dashboard (Next.js)*. GitHub Repository. Retrieved from: <https://github.com/sparktechagency/rollestone>
- [3] SparkTech Agency. (2026). *Rollestone Express App (React Native)*. GitHub Repository. Retrieved from: <https://github.com/sparktechagency/RollestionExpress>
- [4] SparkTech Agency. (2026). *Rollestone Backend API (Laravel)*. GitHub Repository. Retrieved from: <https://github.com/sparktechagency/bus-tricket>
- [5] Otwell, T. (2025). *Laravel 12 Documentation*. Retrieved from <https://laravel.com/docs>
- [6] Facebook Open Source. (2024). *React Native: Learn once, write anywhere*. Retrieved from <https://reactnative.dev>
- [7] Docker Inc. (2024). *Docker Containerization Best Practices*. Retrieved from <https://docs.docker.com>
- [8] Amazon Web Services. (2024). *AWS Well-Architected Framework*. Retrieved from <https://aws.amazon.com/architecture>
- [9] Robusto, C. C. (1957). *The Cosine-Haversine Formula*. The American Mathematical Monthly, 64(1), 38–40.
- [10] Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- [11] HashiCorp. (2025). *Terraform: Infrastructure as Code*. Retrieved from <https://www.terraform.io>
- [12] OWASP Foundation. (2023). *OWASP Top Ten Web Application Security Risks*. Retrieved from <https://owasp.org/www-project-top-ten/>